Reviewer Pack — Minimal Frobenius Monomial Rank and Frege Proof Length Corre…

Entry dc964fc9921d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 06:08:08 UTC

Minimal Frobenius Monomial Rank and Frege Proof Length Correlation

Entry ID: `dc964fc9921d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 06:08:08 UTC

1. Conjecture

Field A (mathematical branch): Frobenius Algebra Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

> For every Boolean formula φ, the minimal Frobenius monomial rank (mfr(φ)) of its associated algebraic structure is linearly correlated with its Frege proof length l(φ), such that mfr(φ) = Θ(l(φ)). Additionally, for any instance φ with a Frege proof depth d(φ), there exists a polynomial-time computable Frobenius algebraic operation that can be applied to the algebraic structure of φ to reduce its Frege proof length by at least half.

Rationale (proposer's reasoning):

> Frobenius algebras provide a framework for studying non-associative operations, which are closely related to the complexity of boolean functions. The conjecture suggests that the structure captured by Frobenius algebras can be used to predict and manipulate the proof length in Frege proofs, providing new insights into proof complexity.

Taxonomy category: `Frobenius-Algebra-Proof-Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d42bbaa9d3d47f94`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between minimal Frobenius monomial rank (mfr(φ)) and Frege proof length (l(φ)) for all Boolean formulas φ is ≥ 0.8, with p-value ≤ 0.05 using 30 random seeds, AND if a polynomial-time Frobenius algebraic operation can halve the proof length of any instance φ with depth d(φ) within <240 seconds for n ≤ 40.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Frobenius Monomial Rank AND Frege Proof Length` - `Frobenius Algebra Theory INTEGRATED WITH Frege Proof Complexity` - `Polynomial-time Computable Frobenius Algebraic Operations FOR Reducing Frege Proof Depth`

Top relevant hits considered: - [s2:2ba520736fa9f9a8a12733c78038a5e442f8a2cc] Automorphisms of Rings and Applications to Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        if n == 1:
            return 'x'
        else:
            left = generate_formula(random.randint(1, n-1))
            right = generate_formula(n - len(left.split('&')) - 1)
            return f'({left}&{right})'

    def frege_proof_length(formula):
        if formula == 'x':
            return 1
        else:
            left, right = formula[2:-1].split('&')
            return 1 + max(frege_proof_length(left), frege_proof_length(right))

    def frobenius_monomial_rank(formula):
        if formula == 'x':
            return 1
        else:
            left, right = formula[2:-1].split('&')
```

```
            return frobenius_monomial_rank(left) + frobenius_monomial_rank(right)

    def reduce_proof_length(formula):
        if formula == 'x':
            return 'x'
        else:
            left, right = formula[2:-1].split('&')
            new_left = reduce_proof_length(left)
            new_right = reduce_proof_length(right)
            if len(new_left.split('&')) > len(new_right.split('&')):
                return f'({new_left}&{right})'
            else:
                return f'({left}&{new_right})'

    n_values = [5, 10, 15, 20, 30, 40]
    instances_tested = 0
    total_mfr = 0
    total_l = 0
    counterexample = ""

    for n in n_values:
        for _ in range(5):
            formula = generate_formula(n)
            mfr = frobenius_monomial_rank(formula)
            l = frege_proof_length(formula)
            instances_tested += 1
            total_mfr += mfr
            total_l += l

            if mfr != l:
                counterexample = f"Formula: {formula}, MFR: {mfr}, L: {l}"
                break

        if counterexample:
            break

    mean_mfr = total_mfr / instances_tested
    mean_l = total_l / instances_tested

    conjecture_holds = abs(mean_mfr - mean_l) < 0.1 * max(mean_mfr, mean_l)

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": (mean_mfr * mean_l - instances_tested * mean_mfr * mean_l / instances_test
                         math.sqrt((instances_tested * mean_mfr**2 - total_mfr**2 / instances_teste
                                    (instances_tested * mean_l**2 - total_l**2 / instances_tested)),
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
```

```
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc20d4e2.py", line 102, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc20d4e2.py", line 63, in run_trial
    formula = generate_formula(n)
              ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc20d4e2.py", line 25, in generate_form
    left = generate_formula(random.randint(1, n-1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc20d4e2.py", line 25, in generate_form
    left = generate_formula(random.randint(1, n-1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc20d4e2.py", line 25, in generate_form
    left = generate_formula(random.randint(1, n-1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc20d4e2.py", line 26, in generate_form
    right = generate_formula(n - len(left.split('&')) - 1)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc20d4e2.py", line 25, in generate_form
    left = generate_formula(random.randint(1, n-1))
                            ^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(1, 0)
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that it was unable to complete the required computations to verify the conjecture. | next: Investigate and fix the error in the test code to ensure it can run to completion. Once fixed, re-run the test with a larger sample size or more iterations to provide stronger evidence for or against the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15698
2	preregistration	ollama_remote	glm4:latest	0	0	14133
3	novelty	ollama_remote	glm4:latest	0	0	8866
4	novelty	ollama_remote	glm4:latest	0	0	9510
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14053
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13124
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18511
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12212
9	judge	ollama_remote	glm4:latest	0	0	12264

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 118371 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/dc964fc9921d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dc964fc9921d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/dc964fc9921d.tar.gz` (if generated)